

Trabajo Práctico Integrador – Programación 2

Food Store – Sistema de Gestión de Pedidos de Comida (Consola + JDBC)

Se propone desarrollar un sistema de consola llamado Food Store, orientado a la gestión de productos de un negocio de comidas mediante una aplicación Java basada en Programación Orientada a Objetos (POO).

El sistema permitirá gestionar categorías, productos, usuarios y pedidos (según el UML provisto), realizando operaciones CRUD completas desde un menú de opciones en consola. El acceso se realizará directamente a través del menú para ejecutar altas, bajas, modificaciones y consultas (no se implementará login).

Como el sistema no contará con base de datos, **toda la información se almacenará en memoria durante la ejecución del programa mediante el uso de Colecciones.**

El desarrollo deberá realizarse tomando como guía un backlog de épicas e historias de usuario. Esto implica transformar cada historia en opciones concretas del menú. Además, se deberán incorporar validaciones de entrada, manejo de excepciones propias y una organización clara de responsabilidades (modelo, servicios/lógica y vista por consola).

Food Store – Sistema de Gestión de Pedidos de Comida (Consola + JDBC) 1

 Alcance de Programación 2.....	2
 Información del Proyecto.....	2
 Objetivos de Aprendizaje.....	3
 Modelo (UML) y Entidades.....	4
 Interfaz de Consola (Menú).....	5
 Arquitectura Recomendada.....	6
<i>Regla general:</i>	6
 Reglas mínimas de negocio (para consola).....	6
 Definición de Completado (DoD)	6
 Criterios de Evaluación (orientativos).....	6
<i>Método de Entrega:</i>	7
• <i>El código fuente completo desarrollado.</i>	8

ANEXO – Historias de Usuario (CRUD en Consola).....	8
Épica 1 – Gestión de Categorías	8
<i>HU-CAT-01 – Listar categorías</i>	<i>8</i>
<i>HU-CAT-02 – Crear categoría.....</i>	<i>8</i>
<i>HU-CAT-03 – Editar categoría.....</i>	<i>9</i>
<i>HU-CAT-04 – Eliminar categoría (baja lógica).....</i>	<i>9</i>
Épica 2 – Gestión de Productos	9
<i>HU-PROD-01 – Listar productos.....</i>	<i>9</i>
<i>HU-PROD-02 – Crear producto</i>	<i>9</i>
<i>HU-PROD-03 – Editar producto</i>	<i>10</i>
<i>HU-PROD-04 – Eliminar producto (baja lógica)</i>	<i>10</i>
Épica 3 – Gestión de Usuarios	10
<i>HU-USR-01 – Listar usuarios.....</i>	<i>10</i>
<i>HU-USR-02 – Crear usuario</i>	<i>10</i>
<i>HU-USR-03 – Editar usuario</i>	<i>11</i>
<i>HU-USR-04 – Eliminar usuario (baja lógica).....</i>	<i>11</i>
Épica 4 – Gestión de Pedidos y Detalles.....	11
<i>HU-PED-01 – Listar pedidos.....</i>	<i>11</i>
<i>HU-PED-02 – Crear pedido con detalles.....</i>	<i>11</i>
<i>HU-PED-03 – Actualizar estado/forma de pago del pedido.....</i>	<i>12</i>
<i>HU-PED-04 – Eliminar pedido (baja lógica).....</i>	<i>12</i>



Alcance de Programación 2

- Toda la interacción ocurre en consola mediante un menú.
- Se aplique POO (encapsulamiento, constructores, herencia, polimorfismo) según el UML.
- Se utilicen **Colecciones** para el almacenamiento y gestión de los objetos en memoria.
- Se implementen **Interfaces** para definir comportamientos comunes.
- Se aplique el **Manejo de Excepciones** para validar errores y evitar cierres inesperados del programa.



Información del Proyecto

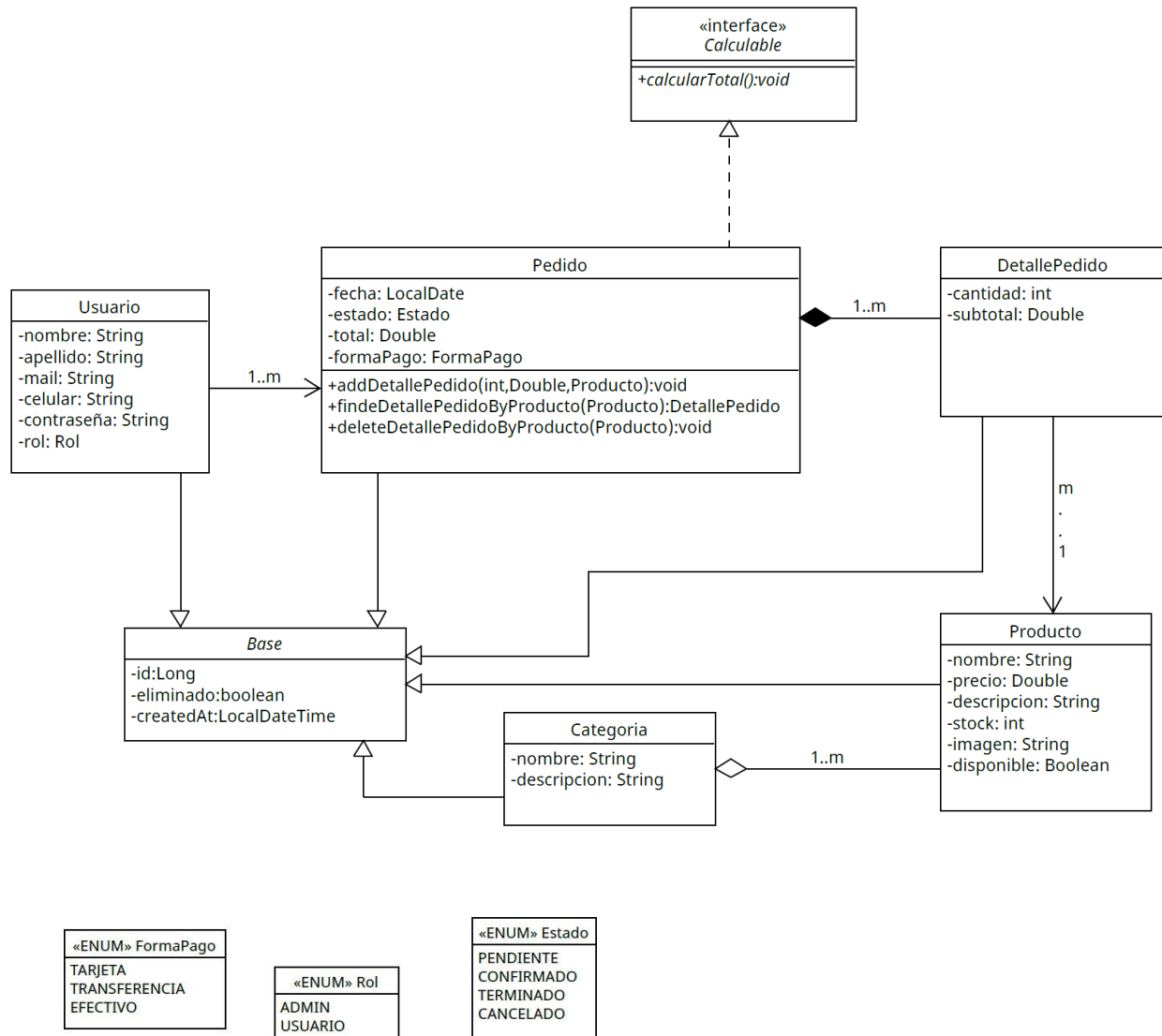
- **Modalidad:** Aplicación de consola (menú).
- **Lenguaje:** Java 21.
- **Almacenamiento:** En memoria (Colecciones).
- **Patrones/Capas:** Entidades + Servicios/Lógica + UI (Main/Menu).

Objetivos de Aprendizaje

Al finalizar el TPI, el proyecto debe evidenciar que el estudiante puede:

- Modelar correctamente clases y relaciones del UML (asociaciones 1..N y N..1).
- Aplicar herencia (clase base) y sobrescritura de métodos (toString, equals).
- Utilizar estructuras de datos dinámicas (Colecciones) para gestionar las entidades.
- Implementar Interfaces para estandarizar comportamientos (ej. Calculable).
- Gestionar errores creando y capturando Excepciones propias (por ejemplo, EntidadNoEncontradaException, StockInvalidoException).

Modelo (UML) y Entidades



El sistema se basa en un diseño orientado a objetos donde varias clases comparten atributos comunes mediante herencia, implementando interfaces para comportamientos específicos:

- **Clase Abstracta Base:** id (Long), eliminado (boolean), createdAt (LocalDateTime). Todas las entidades a continuación heredan de esta clase.
- **Categoria:** hereda de Base + nombre, descripcion (y relación 1:N con Producto).
- **Producto:** hereda de Base + nombre, precio, descripcion, stock, imagen, disponible (y relación N:1 con Categoria).

- **Usuario:** hereda de Base + nombre, apellido, mail (único), celular, contraseña, rol (y relación 1:N con Pedido).
- **Pedido:** hereda de Base + fecha (LocalDate), estado, total, formaPago (y relación de composición 1:N con DetallePedido).
 - *Implementa la interfaz* Calculable (método calcularTotal()).
 - *Métodos propios:* addDetallePedido(int, Double, Producto),
findeDetallePedidoByProducto(Producto),
deleteDetallePedidoByProducto(Producto).
- **DetallePedido:** hereda de Base + cantidad, subtotal (y relación N:1 con Producto).
- **Enums:** Rol (ADMIN, USUARIO), Estado (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO), FormaPago (TARJETA, TRANSFERENCIA, EFECTIVO).
- **Interfaz:** Calculable (define el comportamiento para calcular totales).



Interfaz de Consola (Menú)

El sistema debe iniciar mostrando un menú principal similar al siguiente (el texto puede variar, pero la idea es la misma):

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
```

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir

Seleccione:

Cada opción abre un submenú CRUD. Ejemplo para Categorías:

1. Listar
2. Crear
3. Editar
4. Eliminar

Seleccione:

Pautas importantes del menú:

- El menú debe validar entradas (opciones fuera de rango, ids inexistentes, números mal ingresados).
- Las operaciones deben mostrar mensajes claros (éxito / error) y retornar al menú.
- Para facilitar pruebas, es válido listar elementos antes de pedir un ID (por ejemplo: listar categorías y luego pedir id).



Arquitectura Recomendada

Para mantener el código ordenado y evaluable, se recomienda una estructura por paquetes similar a:

```
src/  
├── integrado/prog2/  
│   ├── Main.java  
│   ├── config/      (ConexionDB, lectura de configuración)  
│   ├── entities/    (clases del UML)  
│   ├── enums/       (Rol, Estado, FormaPago)  
│   └── exception/   (excepciones propias)
```

Regla general:

- Main/Menu: solo interacción con usuario (Scanner) y armado de objetos.
- Service: coordinación de casos de uso (validaciones, reglas de negocio simples).
- Entities: modelo de dominio (POO), con toString() útil para listados en consola.



Reglas mínimas de negocio (para consola)

Además del CRUD, se espera que el sistema respete reglas básicas para que el dominio tenga sentido:

- No permitir crear Producto con precio < 0 o stock < 0 (Lanzar excepción).
- No permitir crear Pedido sin usuario.
- Al crear DetallePedido, la cantidad debe ser > 0.
- Mail de Usuario debe ser único (validar recorriendo la colección e informar error).
- Soft delete: marcar eliminado = true en vez de remover físicamente el objeto de la colección, para mantener el historial.



Definición de Completado (DoD)

- Compila y ejecuta sin errores.
- Permite al menos CRUD de Categoría, Producto, Usuario y Pedido desde consola afectando las colecciones en memoria.
- El código está modularizado (el Main no contiene reglas de negocio complejas).
- Manejo robusto de errores mediante Try-Catch y Excepciones.



Criterios de Evaluación (orientativos)

- POO: correcto modelado del UML, encapsulamiento, constructores, herencia y toString útil.
- Calidad de código: nombres claros, separación por capas, no duplicación.

- Robustez: validaciones, mensajes de error y flujo de menú estable.



Entregables

Contenido de la Entrega

1. Entregables Técnicos (Repositorio GitHub)
 - Proyecto Java completo: Código fuente organizado por paquetes.
 - Archivo README.md: Un documento breve en la raíz indicando cómo ejecutar el proyecto.
 2. Documentación Académica y Técnica (Formato PDF) El documento debe estar correctamente ordenado, con las hojas numeradas y mantener un formato profesional. Debe incluir:
 - Carátula: Nombre de la institución, carrera, nombre de la materia (Programación 2), título del proyecto, datos del estudiante y fecha de entrega.
 - Índice: Tabla de contenidos estructurada con los números de página correspondientes para facilitar la navegación.
 - Marco Teórico: Breve descripción de las tecnologías utilizadas (Java, bases de datos relacionales, etc.) y conceptos clave aplicados.
 - Decisiones Técnicas y Arquitectura: Explicación de cómo se abordó la solución, cómo se organizaron los paquetes, y capturas de pantalla del sistema funcionando.
 - Dificultades y Soluciones: Un breve apartado comentando los principales obstáculos técnicos que surgieron durante el desarrollo y cómo lograron resolverlos.
 - Bibliografía / Webgrafía: Enlaces a la documentación oficial, repositorios o recursos utilizados como apoyo.
 - Links al repositorio del proyecto y link del video explicativo solicitado más abajo.
 3. Video Demostración
 - Grabación de 10 a 15 minutos mostrando el funcionamiento del sistema ya levantado.
- Se debe recorrer obligatoriamente el flujo completo del programa para validar que cumple con los requisitos solicitados.

Método de Entrega:

La entrega del proyecto deberá realizarse **exclusivamente en formato .zip**. El archivo comprimido deberá contener obligatoriamente:

- El **código fuente completo desarrollado**.
- El **documento en formato PDF** confeccionado según las consignas establecidas.

Asimismo, dentro del archivo **README.md** del repositorio se deberá incluir obligatoriamente:

- El **enlace al video demostrativo**, asegurando que cuente con permisos públicos de visualización.
- El **enlace a la documentación en PDF**, o bien el archivo PDF subido directamente en la raíz del repositorio.

No se considerarán entregas incompletas o que no respeten el formato indicado.

ANEXO – Historias de Usuario (CRUD en Consola)

Este anexo funciona como backlog mínimo para guiar el desarrollo del sistema en consola. Cada historia se cumple implementando las opciones del menú y su lógica.

Notas:

- No hay login: se asume un usuario operador que accede directamente a los menús de gestión.
- Todas las bajas son lógicas (soft delete) usando el campo eliminado = true.
- Las interacciones se realizan mediante menús de consola.

Épica 1 – Gestión de Categorías

Historias:

HU-CAT-01 – Listar categorías

Como operador del sistema, quiero listar las categorías registradas, para poder ver cuáles están disponibles para asociar productos.

Criterios de aceptación:

- Al seleccionar “Categorías > Listar”, se muestran solo categorías no eliminadas.
- Cada fila muestra al menos: id, nombre y descripción.
- Si no hay registros, se informa “No hay categorías cargadas”.

HU-CAT-02 – Crear categoría

Como operador del sistema, quiero crear una categoría, para organizar los productos del catálogo.

Criterios de aceptación:

- El menú solicita nombre y descripción (con validación de no vacío).

- Se agrega la categoría a la colección y se informa el id generado.
- No se permite crear dos categorías con el mismo nombre (se valida recorriendo la colección y se informa el error).

HU-CAT-03 – Editar categoría

Como operador del sistema, quiero editar una categoría existente, para mantener actualizada su información.

Criterios de aceptación:

- El sistema permite seleccionar la categoría por id (listando previamente de forma opcional).
- Si el id no existe o está eliminado, se informa y no se modifica nada.
- Se actualiza nombre y/o descripción y se confirma la operación.

HU-CAT-04 – Eliminar categoría (baja lógica)

Como operador del sistema, quiero eliminar lógicamente una categoría, para que deje de estar disponible sin perder historial.

Criterios de aceptación:

- Al seleccionar “Eliminar”, se pide id y confirmación (S/N).
- La operación marca eliminado = true (sin remover físicamente el objeto de la colección).
- La categoría eliminada no vuelve a mostrarse en listados.
- Si la categoría tiene productos asociados, se informa la regla definida por la cátedra (por ejemplo: impedir la baja o permitirla igual).

Épica 2 – Gestión de Productos

Historias:

HU-PROD-01 – Listar productos

Como operador del sistema, quiero listar productos, para consultar sus datos y stock.

Criterios de aceptación:

- Se muestran solo productos no eliminados.
- Cada fila muestra: id, nombre, precio, stock y categoría asociada (al menos el id o nombre).
- Se permite listar por categoría (opcional) o listado general.

HU-PROD-02 – Crear producto

Como operador del sistema, quiero crear un producto y asociarlo a una categoría, para incorporarlo al catálogo.

Criterios de aceptación:

- El menú solicita: nombre, descripción, precio, stock, imagen, estado de disponibilidad

(booleano) y categoría (por id).

- Validaciones: precio ≥ 0 y stock ≥ 0 ; nombre no vacío.
- Si la categoría no existe o está eliminada, no se permite continuar.
- Se agrega el producto a la colección y se informa el id generado.

HU-PROD-03 – Editar producto

Como operador del sistema, quiero editar un producto, para actualizar precio, stock o su categoría.

Criterios de aceptación:

- Se selecciona producto por id.
- Se permite actualizar uno o más campos (manteniendo los anteriores si el usuario no modifica).
- Validaciones: precio ≥ 0 y stock ≥ 0 .
- Si el id no existe o está eliminado, se informa y se cancela.

HU-PROD-04 – Eliminar producto (baja lógica)

Como operador del sistema, quiero eliminar lógicamente un producto, para retirarlo del catálogo sin borrar su información histórica.

Criterios de aceptación:

- Se pide id y confirmación.
- Se marca eliminado = true.
- No se muestra en listados posteriores.
- Si el producto está referenciado en detalles de pedidos, no debe romperse la integridad de los datos (no remover de la colección).

Épica 3 – Gestión de Usuarios

Historias:

HU-USR-01 – Listar usuarios

Como operador del sistema, quiero listar usuarios, para consultar su información de contacto.

Criterios de aceptación:

- Se listan solo usuarios no eliminados.
- Se muestra: id, nombre, apellido, mail, rol (si aplica en el UML).
- Si no hay usuarios, se informa adecuadamente.

HU-USR-02 – Crear usuario

Como operador del sistema, quiero crear un usuario, para poder asociarlo luego a pedidos.

Criterios de aceptación:

- El menú solicita: nombre, apellido, mail, celular (según UML).
- Validación de mail no vacío y formato básico (opcional).
- El mail debe ser único: si ya existe, se informa el error y se solicita otro.
- Se persiste el usuario y se informa el id generado.

HU-USR-03 – Editar usuario

Como operador del sistema, quiero editar un usuario, para corregir o actualizar sus datos.

Criterios de aceptación:

- Selección por id.
- Si el id no existe o está eliminado, se informa.
- Si se modifica el mail, se valida unicidad.
- Se confirma actualización al finalizar.

HU-USR-04 – Eliminar usuario (baja lógica)

Como operador del sistema, quiero eliminar lógicamente un usuario, para que no pueda ser usado en nuevos pedidos.

Criterios de aceptación:

- Se pide id y confirmación.
- Se marca eliminado=true.
- No aparece en listados ni puede seleccionarse al crear pedidos.
- Pedidos existentes del usuario deben seguir pudiendo consultarse (historial).

Épica 4 – Gestión de Pedidos y Detalles

Historias:

HU-PED-01 – Listar pedidos

Como operador del sistema, quiero listar pedidos, para consultar su estado y total.

Criterios de aceptación:

- Se listan solo pedidos no eliminados.
- Se muestra: id, usuario asociado, estado, forma de pago, total y fecha.
- Se permite filtrar por usuario (opcional).

HU-PED-02 – Crear pedido con detalles

Como operador del sistema, quiero crear un pedido y cargar sus detalles (productos y cantidades), para registrar una compra completa.

Criterios de aceptación:

- El menú permite seleccionar usuario por id (debe existir y no estar eliminado).
- Permite agregar 1..N detalles utilizando obligatoriamente el método addDetallePedido(...) de la clase Pedido.
- Se calcula el subtotal de cada detalle al agregarlo (cantidad * precio del producto).
- Se debe utilizar obligatoriamente la interfaz Calculable y su método calcularTotal() para establecer el total final del Pedido.
- Manejo de errores: si ocurre una excepción al agregar un detalle (por ejemplo, por falta de stock), se debe capturar el error y cancelar la creación del pedido en memoria para no dejar datos inconsistentes.

HU-PED-03 – Actualizar estado/forma de pago del pedido

Como operador del sistema, quiero actualizar el estado y/o forma de pago de un pedido, para reflejar su avance.

Criterios de aceptación:

- Se selecciona pedido por id.
- Se permite cambiar estado (enum) y forma de pago (enum) mediante opciones numéricas validadas.
- No se permite pasar a estados inválidos si la cátedra define reglas (opcional).
- Se actualizan los datos en memoria y se informa el éxito.

HU-PED-04 – Eliminar pedido (baja lógica)

Como operador del sistema, quiero eliminar lógicamente un pedido, para ocultarlo sin perder auditoría.

Criterios de aceptación:

- Se pide id y confirmación.
- Se marca eliminado=true en pedido (y, si corresponde, en sus detalles).
- El pedido eliminado no se muestra en listados.
- No deben quedar las colecciones en memoria en estado inconsistente.